

Configuring an AWS Client VPN

To Begin with the Lab

Summary of the Lab

This lab demonstrates how to configure an AWS Client VPN using certificate-based (X.509) mutual authentication. It begins by creating a Certificate Authority, server, and client certificates with Easy-RSA on an Amazon Linux EC2 instance, then importing them into AWS Certificate Manager. A VPC with separate application and VPN subnets, route tables, security groups, and a private EC2 instance is configured. An AWS Client VPN endpoint is created, associated with the target subnet, and authorized for user access. The VPN profile is imported into the OpenVPN client to verify secure private connectivity. Finally, Split Tunnel is enabled and the OpenVPN configuration is updated to ignore unsupported AWS DNS options, restoring internet access while maintaining VPN connectivity.

Note: The objective of this lab is to configure an **AWS Client VPN** so that a client machine (your laptop) can securely access an EC2 instance inside a private subnet using its **Private IP Address**. Authentication is performed using **X.509 certificates**, and the OpenVPN client is used to establish the VPN connection.

Create a EC2 instance with Amazon Linux to create the certificate.

First update the package metadata and installed packages, then install Git.

```
sudo dnf update -y  
sudo dnf install git -y
```

To verify the installation:

```
git --version
```

Clone the OpenVPN repo from [github](https://github.com/OpenVPN/easy-rsa)

```
git clone https://github.com/OpenVPN/easy-rsa.git
```

after cloning, go the directory

```
cd easy-rsa/easyrsa3
```

This PKI will be used to create the Certificate Authority (CA), server certificate, and client certificate required for AWS Client VPN authentication.

run the command

```
./easyrsa init-pki
```

```
[ec2-user@ip-172-31-19-222 easyrsa3]$ ./easyrsa init-pki
Notice
-----
'init-pki' complete; you may now create a CA or requests.

Your newly created PKI dir is:
* /home/ec2-user/easy-rsa/easyrsa3/pki

[ec2-user@ip-172-31-19-222 easyrsa3]$
```

Then run the command

```
./easysrsa build-ca nopass
```

Give any name

[illegible]

Then run this command

```
./easysrsa --san=DNS:server build-server-full server nopass
```

And confirm it but typing

yes

[illegible]

Then run this command

```
./easysrsa build-client-full client1.domain.tld nopass
```

Type

yes

```
[ec2-user@ip-172-31-19-222 easyrsa3]$ ./easyrsa build-client-full client1.domain.tld nopass
.....
-----
Notice
-----
Private-Key and Public-Certificate-Request files created.
Your files are:
* req: /home/ec2-user/easy-rsa/easyrsa3/pki/reqs/client1.domain.tld.req
* key: /home/ec2-user/easy-rsa/easyrsa3/pki/private/client1.domain.tld.key
You are about to sign the following certificate:
Requested CN:      'client1.domain.tld'
Requested type:    'client'
Valid for:        '825' days
subject=
  commonName      = client1.domain.tld
Type the word 'yes' to continue, or any other input to abort.
Confirm requested details: yes
```

Create a new directory to keep all the certificates and key together

```
mkdir ~/demo
cp pki/ca.crt ~/demo/
cp pki/issued/server.crt ~/demo/
cp pki/private/server.key ~/demo/
cp pki/issued/client1.domain.tld.crt ~/demo/
cp pki/private/client1.domain.tld.key ~/demo/
```

```
[ec2-user@ip-172-31-19-222 demo]$ ls
ca.crt  client1.domain.tld.crt  client1.domain.tld.key  server.crt  server.key
[ec2-user@ip-172-31-19-222 demo]$
```

Import Certificates into AWS Certificate Manager (ACM)

Make sure you are in the same AWS Region where the VPN will be created.

Verify that both certificates are successfully imported.

Import the server certificate

```
aws acm import-certificate --certificate fileb://server.crt --private-key fileb://server.key --
certificate-chain fileb://ca.crt --region us-east-1
```

Import the client certificate

```
aws acm import-certificate --certificate fileb://client1.domain.tld.crt --private-key
fileb://client1.domain.tld.key --certificate-chain fileb://ca.crt --region us-east-1
```

```
[ec2-user@ip-172-31-19-222 demo]$ aws acm import-certificate --certificate fileb://server.crt --private-key fileb://server.key --certificate-chain fileb://ca.crt --
region us-east-1
{
  "CertificateArn": "arn:aws:acm:us-east-1:463646775279:certificate/d199b904-fa65-4ecf-be3d-32f7ed5f602a"
}
[ec2-user@ip-172-31-19-222 demo]$ aws acm import-certificate --certificate fileb://client1.domain.tld.crt --private-key fileb://client1.domain.tld.key --certificate
-chain fileb://ca.crt --region us-east-1
{
  "CertificateArn": "arn:aws:acm:us-east-1:463646775279:certificate/3d1317c2-392d-4e61-b129-61d6475d7bca"
}
[ec2-user@ip-172-31-19-222 demo]$
```

When you open your ACM on AWS console, you will see this

Certificates (4)					
Search certificates					
	Certificate ID	Common name	Type	Status	In us
☐	3d1317c2-392d-4e61-b129-61d6475d7bca	client1.domain.tld	Imported	Issued	No
☐	d199b904-fa65-4ecf-be3d-32f7ed5f602a	server	Imported	Issued	No

Create a new VPC named **cf-demo**.

Assign the CIDR block **10.10.0.0/16**.

VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only
 ☐ VPC and more

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

cf-demo

IPv4 CIDR block [Info](#)
☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR
10.10.0.0/16

CIDR block size must be between /16 and /28.

Use the CIDR block **10.10.0.0/24**.

This subnet will host the EC2 instance that clients will access through the VPN.

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

cf-demo-app-subnet

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

United States (N. Virginia) / use1-az4 (us-east-1a)

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.10.0.0/16

IPv4 subnet CIDR block

10.10.0.0/24 256 IPs

Name it **cf-demo-clientVPN-subnet**.

Assign the CIDR block **10.10.100.0/24**.

This subnet will later become the **Target Network** for the Client VPN Endpoint.

Subnet 2 of 2

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

cf-demo-clientVPN-subnet

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

United States (N. Virginia) / use1-az4 (us-east-1a)

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.10.0.0/16

IPv4 subnet CIDR block

10.10.100.0/24 256 IPs

Create separate Route Tables for both subnets.

Associate one Route Table with the **Application Subnet** and another with the **Client VPN Subnet**.

No additional routes are required at this stage.

Route table settings

Name - optional

Create a tag with a key of 'Name' and a value that you specify.

app-rt

VPC

The VPC to use for this route table.

vpc-0f297d0d2d836a6e7 (cf-demo)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name

Value - optional

Q app-rt

Remove

Add new tag

You can add 49 more tags.

Cancel

Create route table

rtb-0db476dbfd89965a5 / app-rt

Actions

Details Info

Route table ID

rtb-0db476dbfd89965a5

VPC

vpc-0f297d0d2d836a6e7 | cf-demo

Main

No

Owner ID

463646775279

Explicit subnet associations

subnet-0ff412940909ad5a6 / cf-demo-app-subnet

Edge associations

-

Routes

Subnet associations

Edge associations

Route propagation

Tags

Explicit subnet associations (1)

Edit subnet associations

Find subnet association

< 1 >

Name Subnet ID IPv4 CIDR IPv6 CIDR

cf-demo-app-subnet subnet-0ff412940909ad5a6 10.10.0.0/24 -

For VPN

rtb-098e6f906121021e4 / vpn-rt

Actions

Details Info

Route table ID

rtb-098e6f906121021e4

VPC

vpc-0f297d0d2d836a6e7 | cf-demo

Main

No

Owner ID

463646775279

Explicit subnet associations

subnet-070c1a3947a951951 / cf-demo-clientVPN-subnet

Edge associations

-

Routes

Subnet associations

Edge associations

Route propagation

Tags

Explicit subnet associations (1)

Edit subnet associations

Find subnet association

< 1 >

Name Subnet ID IPv4 CIDR IPv6 CIDR

cf-demo-clientVPN-subnet subnet-070c1a3947a951951 10.10.100.0/24 -

Create a new Security Group named **Demo-ClientVPN-SG**.

Associate it with **Demo-VPC**.

Leave the default outbound rule that allows all outbound traffic.

No inbound rules are required for this Security Group.

Basic details

Security group name [Info](#)
Demo-ClientVPN-SG
Name cannot be edited after creation.

Description [Info](#)
for vpn connectivity

VPC [Info](#)
vpc-0f297d0d2d836a6e7 (cf-demo)

Inbound rules [Info](#)

This security group has no inbound rules.

[Add rule](#)

Launch an EC2 instance inside the **Application Subnet**.

Disable **Auto-assign Public IP** because the instance should remain private.

This EC2 instance will be used to verify VPN connectivity.

Name and tags [Info](#)

Name
cf-demo-app [Add additional tags](#)

Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Q Search our full catalog including 1000s of application and OS images

Recents My AMIs **Quick Start**

Amazon Linux macOS Ubuntu Windows Red Hat SUSE Linux Debian

[Browse more AMIs](#)
Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.18 AMI
ami-06067086cf86c58e6

Number of instances [Info](#)
1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.12...[read more](#)
ami-06067086cf86c58e6

Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

Network Setting for this ec2

Network settings [Info](#)

VPC - required [Info](#)
vpc-0f297d0d2d836a6e7 (cf-demo)

Subnet [Info](#)
subnet-0ff412940909ad5a6 (cf-demo-app-subnet) [Create new subnet](#)

VPC: vpc-0f297d0d2d836a6e7 Owner: 463646775279
Availability Zone: us-east-1a (us-east-1a) Zone type: Availability Zone
IP addresses available: 250 CIDR: 10.10.0.0/24

Auto-assign public IP [Info](#)
Disable

Firewall (security groups) [Info](#)

Number of instances [Info](#)
1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.12...[read more](#)
ami-06067086cf86c58e6

Virtual server type (instance type)
t2.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Navigate to **VPC → Client VPN Endpoints**.

Click **Create Client VPN Endpoint**.

Provide a suitable name such as **Demo Client VPN Endpoint**.

Create client VPN endpoint [Info](#)

Create a client VPN endpoint to enable access to networks over a TLS VPN session.

Choose setup method

☒ Standard

Manual customization to configure all networking, authentication, and advanced settings.

☐ Quick start

Use recommended defaults with configurations that can be changed after the endpoint is created.

Details

Name - optional

Creates a tag with the key set to Name and the value set to the specified string.

Name must be 1 to 256 characters, and must contain only letters, digits, white space, and the following special characters: _ . : / = + @ -

Description - optional

A brief description of the client VPN endpoint.

Select the **VPC** created earlier.

Attach the **Demo-ClientVPN-SG** Security Group to the VPN Endpoint.

Network infrastructure [Info](#)

Select how to connect your Client VPN endpoint to your network infrastructure.

Association type

Select how to connect your Client VPN endpoint to your network infrastructure.

☒ Virtual Private Cloud (VPC)

Direct attachment to a specific VPC with security group controls.

☐ Transit Gateway (TGW)

Routing hub for scalable multi-VPC connectivity without peering.

☐ None

Configure association type later.

VPC ID

Security group IDs - optional

Security groups define the allowed inbound and outbound traffic rules for the Client VPN endpoint. If not specified, the default security group is applied.

Select security groups

Choose **IPv4** as the address family.

Specify the **Client CIDR Range**.

Use **10.20.0.0/16** as the Client CIDR block.

Ensure that this CIDR range does **not overlap** with either your local network or the VPC CIDR range.

IP address

Endpoint IP address type

The type of connection between the VPN client and server endpoint.

☒ IPv4

☐ IPv6

☐ Dual stack

Traffic IP address type

The type of traffic passing through the VPN tunnel.

☒ IPv4

☐ IPv6

☐ Dual stack

Client IPv4 CIDR [Info](#)

The IP address range, in CIDR notation, from which client IP addresses are allocated when the traffic type is IPv4.

Client IPv4 CIDR must be between /12 and /22, and can't overlap with target network, VPC address range, or routes.

Configure the authentication method.

Select **Mutual Authentication** using certificates.

Choose the **Server Certificate** imported into ACM.

Configure **Certificate-Based Client Authentication** and select the same Certificate Authority.

Authentication [info](#)

Server certificate ARN
The server certificate must be provisioned with or imported into AWS Certificate Manager (ACM).

Authentication options
Choose one or a combination of authentication methods to use.

☒ Use mutual authentication
☐ Use user-based authentication

Client certificate ARN [info](#)

Keep the remaining settings at their default values.

Choose **TCP** as the transport protocol.

▼ **Connection protocol - optional**

Tunnel mode [info](#)
Choose how client traffic is routed through the VPN connection.

☒ **Full-tunnel**
All client traffic is routed through the VPN tunnel.

☐ Split-tunnel
Only traffic matching client VPN endpoint routes is routed through VPN tunnel.

Transport protocol [info](#)
Transport protocol used by the TLS sessions.

☐ UDP
☒ **TCP**

VPN port
AWS client VPN supports ports 443 and 1194 for both TCP and UDP.

Click **Create Client VPN Endpoint**.

Wait until the endpoint status changes to **Available**.

✓ You successfully created cvpn-endpoint-0678c143f44e4a0e6 / Demo-Client-VPN-endpoint. ✕

Client VPN endpoints (1/1) [info](#) [Actions](#) [Client downloads](#) [Download client configuration](#) [Create client VPN endpoint](#)

Name	Client VPN endpoint ID	State	VPC ID	Transit Gateway ID
Demo-Client-VPN-endpoint	cvpn-endpoint-0678c143f44e4a0e6	Pending-associate	vpc-0f297d0d2d836a6e7	-

Open the newly created Client VPN Endpoint.

Navigate to **Target Network Associations**.

Click **Associate Target Network**.

cvpn-endpoint-0678c143f44e4a0e6 / Demo-Client-VPN-endpoint

[Details](#) [Target network associations](#) [Security groups](#) [Authorization rules](#) [Route table](#) [Connections](#) [Tags](#)

Target network associations (1) [info](#) [Disassociate](#) [Associate target network](#)

Association ID	State	Network ID	Security groups	Endpoint ID
----------------	-------	------------	-----------------	-------------

Choose **cf-demo-clientVPN-subnet** as the Target Network.

Create the association.

Wait several minutes until the association becomes **Available**.

☰ VPC > Client VPN endpoints > cvpn-endpoint-0678c143f44e4a0e6 > Associate target network ⓘ

Associate target network ⓘ

A target network is a subnet in a VPC. You associate a subnet in an Availability Zone to the client VPN endpoint. You can associate one subnet per Availability Zone. You can associate subnets in one VPC to a client VPN endpoint.

Details

Client VPN endpoint ID
cvpn-endpoint-0678c143f44e4a0e6

VPC ID
vpc-0f297d0d2d836a6e7

Subnet ID
subnet-070c1a3947a951951 (cf-demo-clientVPN-subnet)

Cancel Associate target network

Open the **Authorization Rules** tab.

Create a new Authorization Rule.

cvpn-endpoint-0678c143f44e4a0e6 / Demo-Client-VPN-endpoint

Details Target network associations Security groups **Authorization rules** Route table Connections Tags

Authorization rules ⓘ

Find authorization rule by attribute or tag

Endpoint ID	State	Description	Group ID	Access all	Destination
No authorization rules					
You do not have any authorization rules in this region					

Create authorization rule

Specify the destination network as either the **Application Subnet CIDR** or the entire **VPC CIDR**.

Allow access for **All Users**.

Save the rule.

Add authorization rule ⓘ

Add authorization rules to grant clients access to the networks.

Details

Client VPN endpoint ID
cvpn-endpoint-0678c143f44e4a0e6

Destination network to enable access
The IP address, in CIDR notation, of the destination network.
10.10.0.0/16

Grant access to:
☒ Allow access to all users
☐ Allow access to users in a specific access group

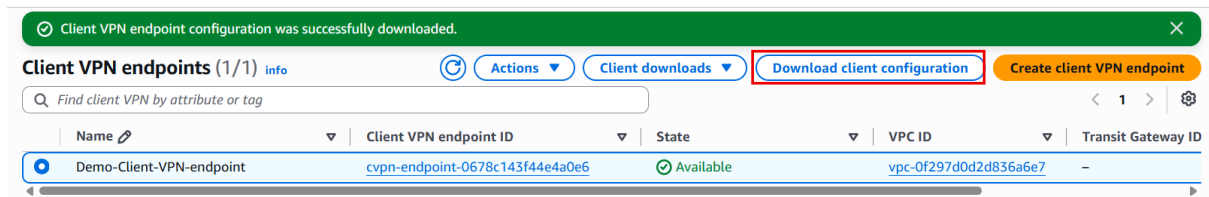
Description - optional
A brief description of the authorization rule.
description

Cancel Add authorization rule

Download the **Client VPN Configuration File** from the AWS Console.

Save the configuration file on your local machine.

This file contains all the settings required by the VPN client.



Open the downloaded configuration file.

Open with notepad

Add the paths to the **Client Certificate** and **Client Private Key**.

Save the updated configuration file.

Note it's better to place all the three file inside one folder.

```
</ca>

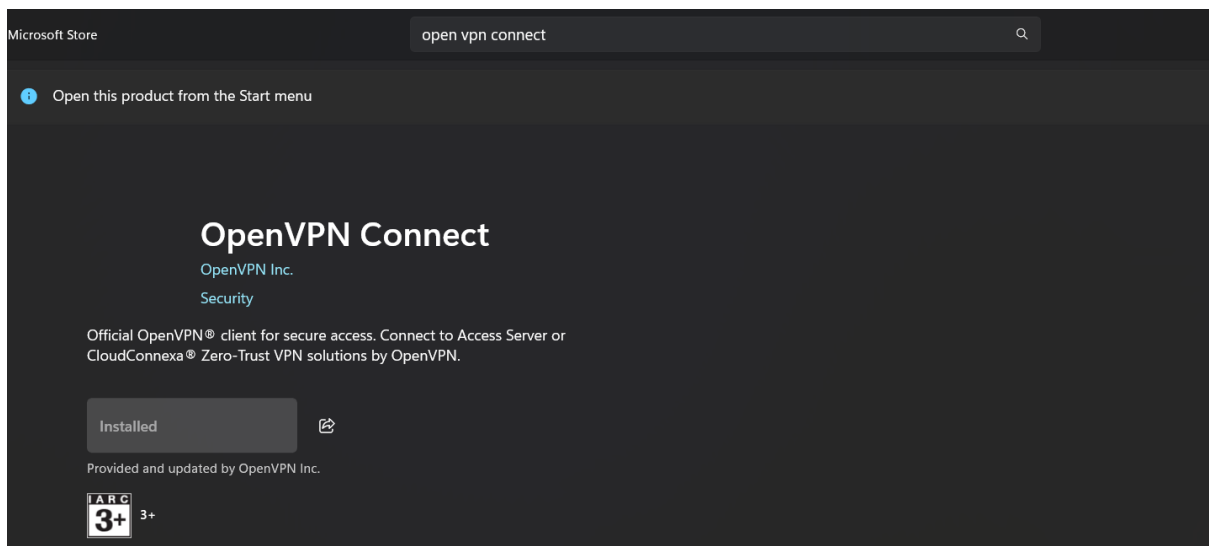
cert client1.domain.tld.crt
key client1.domain.tld.key

reneg-sec 0

verify-x509-name server name
```

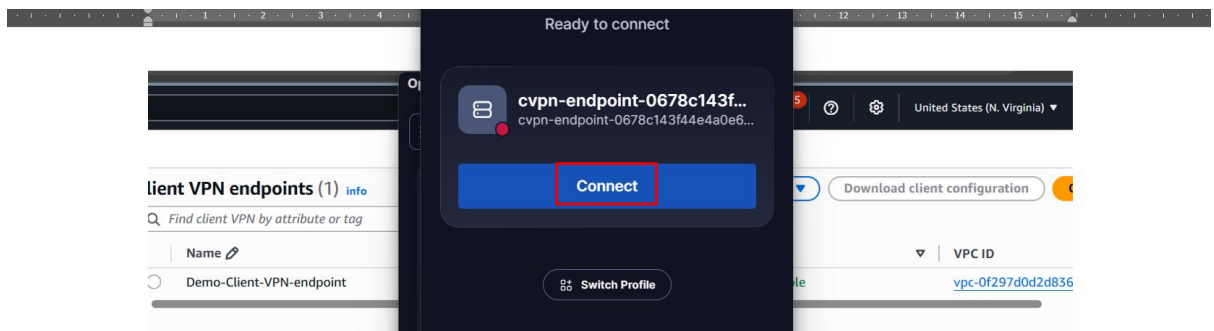
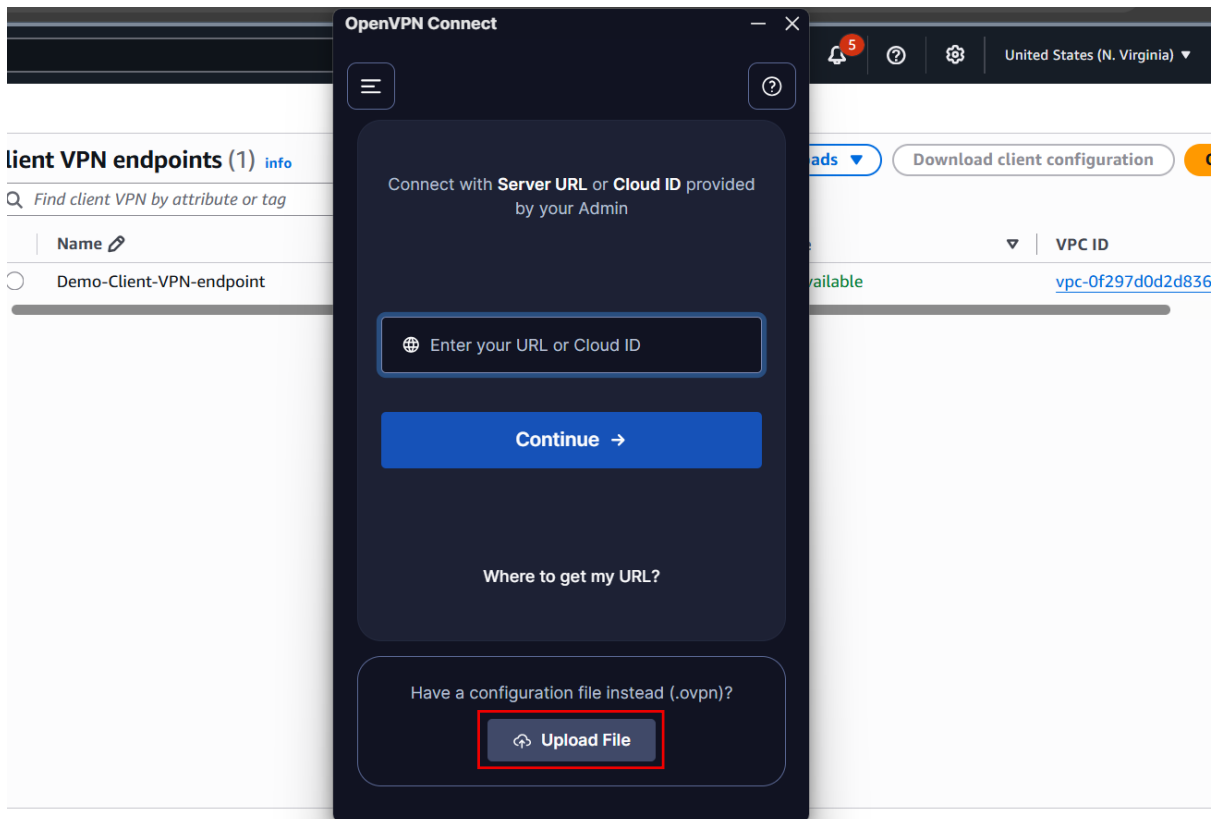
Install the **OpenVPN Client** on your local machine.

You can download it from the Microsoft store

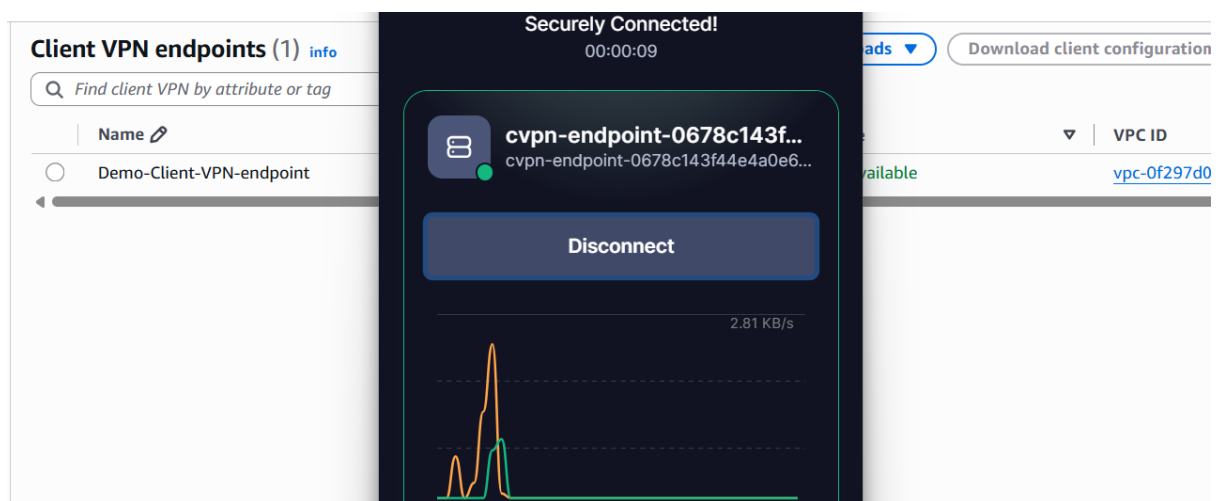


Import the modified VPN configuration file into the VPN client.

Verify that the VPN profile appears successfully.



Connect to the VPN using the imported profile.



After the connection is established, verify that your laptop receives an IP address from the **10.20.0.0/16** Client CIDR range.

Ping the EC2 instance using its Private IP Address.

A successful ping confirms that the AWS Client VPN has been configured correctly.

```
C:\Users\amanr>ping 10.10.0.142

Pinging 10.10.0.142 with 32 bytes of data:
Reply from 10.10.0.142: bytes=32 time=475ms TTL=126
Reply from 10.10.0.142: bytes=32 time=480ms TTL=126
Reply from 10.10.0.142: bytes=32 time=481ms TTL=126
Reply from 10.10.0.142: bytes=32 time=462ms TTL=126

Ping statistics for 10.10.0.142:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 462ms, Maximum = 481ms, Average = 474ms
```

Also, we are not connected to internet.

```
C:\Users\amanr>ping google.com
Ping request could not find host google.com. Please check the name and try again.

C:\Users\amanr>
```

Disconnect the VPN connection from your local machine.

Open **Client VPN Endpoints** and select the VPN Endpoint that was created in the previous lab.

Click **Actions** → **Modify Client VPN Endpoint**.

Enable the **Split Tunnel** option.

Save the changes to update the VPN Endpoint configuration.

The screenshot shows the 'Connection protocol - optional' section in the AWS Management Console. Under 'Tunnel mode', the 'Split-tunnel' option is selected with a radio button. Below it, the 'VPN port' is set to 443 in a dropdown menu. The 'Full-tunnel' option is unselected.

Now you can see internet started working

```

C:\Users\amanr>ping google.com
Ping request could not find host google.com. Please check the name and try again.

C:\Users\amanr>ping google.com

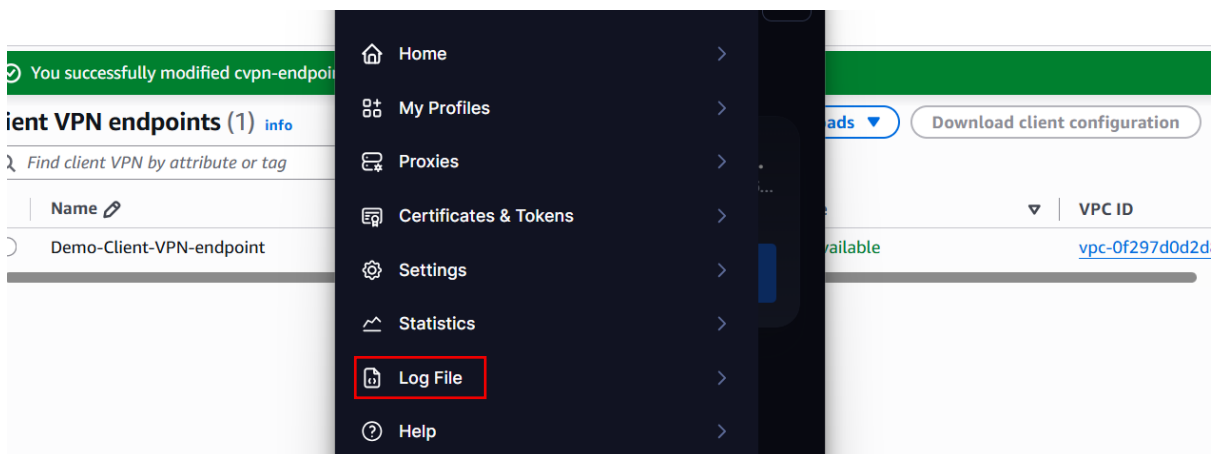
Pinging google.com [142.251.223.238] with 32 bytes of data:
Reply from 142.251.223.238: bytes=32 time=17ms TTL=115
Reply from 142.251.223.238: bytes=32 time=18ms TTL=115
Reply from 142.251.223.238: bytes=32 time=17ms TTL=115
Reply from 142.251.223.238: bytes=32 time=17ms TTL=115

Ping statistics for 142.251.223.238:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 17ms, Maximum = 18ms, Average = 17ms

```

Open the log file of the OpenVPN client.

Review the connection logs and identify the DNS-related error indicating that the client does not recognize the AWS-specific **DNS Route Option**.



Add the required configuration line to ignore the unsupported DNS option that AWS sends to the OpenVPN client.

Save the configuration file after making the change.

Note: This step is generally required only when using the **OpenVPN Client**. If you are using the **AWS VPN Client**, this modification is usually not necessary.

pull-filter ignore "dhcp-option DOMAIN-ROUTE"

```

client
dev tun
proto tcp
remote cvpn-endpoint-0678c143f44e4a0e6.prod.clientvpn.us-east-1.amazonaws.com 443
remote-random-hostname
pull-filter ignore "dhcp-option DOMAIN-ROUTE"
resolv-retry infinite
nobind
remote-cert-tls server
cipher AES-256-GCM
verb 3
<ca>
-----BEGIN CERTIFICATE-----

```

Import the updated VPN configuration file into the VPN client again.

Reconnect to the AWS Client VPN Endpoint.

Verify that connectivity to the EC2 instance still works.

Ping the **Private IP Address** of the EC2 instance to ensure that the VPN connection is functioning correctly after importing the updated configuration.

Ping the public IP address **8.8.8.8** again

```
C:\Users\amanr>ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=18ms TTL=115
Reply from 8.8.8.8: bytes=32 time=17ms TTL=115
Reply from 8.8.8.8: bytes=32 time=20ms TTL=115
Reply from 8.8.8.8: bytes=32 time=17ms TTL=115

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 17ms, Maximum = 20ms, Average = 18ms

C:\Users\amanr>
```